

Web 云盘 — 从 v1 到完全体：分布式架构重构指南

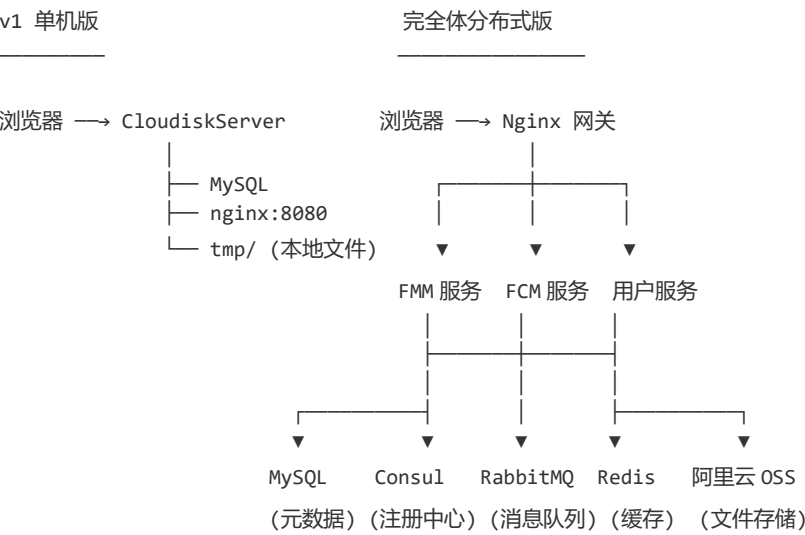
- > 适用场景：毕设、项目经验、校招简历
 - > 前置条件：已理解 v1 单机版 (Workflow + wfrest + MySQL + nginx)
 - > 目标：将单体云盘重构为**微服务分布式云盘系统**
-

目录

1. [架构演进全景图](#一架构演进全景图)
2. [v1 单机版回顾（5 分钟速览）](#二 v1-单机版回顾)
3. [第一层：文件分块上传 + Redis 进度追踪](#三第一层文件分块上传--redis)
4. [第二层：阿里云 OSS — 分布式文件存储](#四第二层阿里云-oss)
5. [第三层：Docker — 容器化部署一切](#五第三层 docker-容器化)
6. [第四层：RabbitMQ — 消息队列异步解耦](#六第四层 rabbitmq-消息队列)
7. [第五层：protobuf + SRPC — 微服务 RPC 通信](#七第五层 protobuf--srpc)
8. [第六层：Consul — 服务注册与发现](#八第六层 consul-服务注册发现)
9. [完全体架构全景](#九完全体架构全景)
10. [毕设级工程实施路线图](#十毕设级工程实施路线图)
11. [关键技术决策与对比](#十一关键技术决策与对比)

12. [可用资源清单](#十二可用资源清单)

一、架构演进全景图



核心变化:

维度	v1	完全体
架构	单体进程	微服务集群
文件存储	本地 tmp/ 目录	阿里云 OSS（分布式对象存储）
大文件处理	整体上传（会超时）	分块上传 + 断点续传
服务通信	无（单进程内部调用）	protobuf + SRPC（远程过程调用）
异步解耦	Workflow Series 链	RabbitMQ 消息队列
服务发现	无	Consul（健康检查 + 负载均衡）
缓存	无	Redis（Token、上传进度、分布式锁）
部署	手动编译运行	Docker 容器化
文件共享	无	RBAC 权限控制 + 链接分享

限速	无	按 VIP 等级分配 FCM 节点数
----	---	--------------------

二、v1 单机版回顾

请求 → wfrest 路由 → Workflow 异步任务 → MySQL → 响应

└─ Series 任务链 (push_back 串联异步任务)

v1 的核心能力（你已经掌握的）：

```
// 1. 异步 MySQL 任务
auto mysqlTask = WFTaskFactory::create_mysql_task(url, retry, callback);
mysqlTask->get_req()->set_query(sql);
series->push_back(mysqlTask); // 任务链：一个接一个执行

// 2. Lambda 值捕获（异步回调的安全保障）
[=](WFMysqlTask *task) { ... } // = 拷贝, & 引用（异步时必须拷贝）

// 3. 密码加密 & Token 生成
string encodedPwd = crypt(password.c_str(), salt.c_str()); // DES
string token = MD5(salt + username) + timestamp;          // 46 字符

// 4. SHA1 文件哈希
SHA1_Init → 循环 SHA1_Update → SHA1_Final → 40 字符 hex
```

v1 的架构缺陷（为什么需要重构）：

- 文件存本地磁盘，机器挂了数据全丢
- 大文件 (>100MB) 上传会超时，无法断点续传

- 所有逻辑挤在一个进程，改一行要全量重部署
 - 没有服务发现，加机器需要改代码
 - 没有权限系统，无法文件共享
 - 单点故障，MySQL 或 服务挂了全部不可用
-

三、第一层：文件分块上传 + Redis

3.1 为什么需要分块上传

一个 500MB 的视频文件，HTTP 请求持续传输 30 分钟。一旦网络断了，全白传。

分块上传的思路：

大文件 → 切成 N 个 block（每块 1MB）→ 逐块上传 → 服务端记录进度 → 全部完成 → 合并

断网了？只重新传未完成的分块。

3.2 分块上传的三个接口

POST /file/mupload/init	- 初始化上传，返回 uploadID
POST /file/mupload/uppart	- 上传一个分块（可并发）
POST /file/mupload/complete	- 通知服务端合并分块

3.3 Redis 存储上传进度

Redis 是内存 KV 数据库，用 **HSET**（哈希表）跟踪上传：

```
HSET uploadID_xxx chunkcount 6      ← 总分块数
HSET uploadID_xxx filehash a1b2c3 ← 文件 SHA1
HSET uploadID_xxx filesize 524288000 ← 文件大小
HSET uploadID_xxx chkidx_0 1        ← 第 0 块已上传
HSET uploadID_xxx chkidx_1 1        ← 第 1 块已上传
HSET uploadID_xxx chkidx_2 1
...
```

3.4 Workflow 操作 Redis

```
// Redis 写操作 - HSET
auto redisTask = WFTaskFactory::create_redis_task(
    "redis://localhost", 0, nullptr);
redisTask->get_req()->set_request("HSET",
    {"uploadID_xxx", "chunkcount", "6"});
redisTask->start(); // fire-and-forget

// Redis 读操作 - HGET (需要回调，因为需要读取结果)
auto redisTask = WFTaskFactory::create_redis_task(
    "redis://localhost", 0, [&](WFRedisTask *task) {
        protocol::RedisValue value;
        task->get_resp()->get_result(value);
        string filehash = value.string_value(); // 获取查到的值
        // 后续处理...
    });
redisTask->get_req()->set_request("HGET", {"uploadID_xxx", "filehash"});
series_of(serverTask)->push_back(redisTask); // 挂到任务链

// 检查上传进度 - HGETALL
redisTask->get_req()->set_request("HGETALL", {"uploadID_xxx"});
// 返回值是数组: [key1, val1, key2, val2, ...]
// 遍历，统计 chkidx_ 前缀的 key 数量 = 已上传块数
// 对比 chunkcount 值 = 总分块数
// 相等 → 上传完成，开始合并
```

3.5 分块上传完整流程代码

```
// === 接口 1: 初始化 ===
// POST /file/mupload/init
// Body: {"filename":"video.mp4", "filesize":524288000, "filehash":"a1b2..."}
if (path == "/file/mupload/init" && method == "POST") {
    // 1. 解析 JSON 请求体
    const void *body; size_t size;
    req->get_parsed_body(&body, &size);
    Json fileInfo = Json::parse(static_cast<const char *>(body));

    string filename = fileInfo["filename"];
    int filesize = fileInfo["filesize"];
    string filehash = fileInfo["filehash"];

    // 2. 生成唯一 uploadID
    string uploadID = "upload_" + to_string(time(nullptr))
        + "_" + to_string(rand());

    // 3. 计算分块信息 (每块 1MB)
    int chunksize = 1024 * 1024; // 1MB
    int chunkcount = (filesize % chunksize == 0)
        ? filesize / chunksize
        : filesize / chunksize + 1;

    // 4. 将分块元信息存入 Redis
    vector<vector<string>> argsVec = {
        {uploadID, "chunkcount", to_string(chunkcount)},
        {uploadID, "filehash", filehash},
        {uploadID, "filesize", to_string(filesize)}
    };
    for (auto &args : argsVec) {
        auto t = WFTaskFactory::create_redis_task(
            "redis://localhost", 0, nullptr);
        t->get_req()->set_request("HSET", args);
        t->start();
    }

    // 5. 返回 uploadID 给客户端
    Json respJson;
    respJson["uploadID"] = uploadID;
    respJson["chunkcount"] = chunkcount;
    respJson["chunksize"] = chunksize;
    resp->append_output_body(respJson.dump());
}

// === 接口 2: 上传一个分块 ===
// POST /file/mupload/uppart?uploadID=xxx&chkidx=2
// Body: 分块的二进制数据
else if (path == "/file/mupload/uppart" && method == "POST") {
    // 1. 从 URL 解析 uploadID 和 chkidx
    // query = "uploadID=xxx&chkidx=2"
    string uploadID = /* 解析 query */;
```

```

string chkidx = /* 解析 query */;

// 2. 从 Redis 获取 filehash (用于建目录)
auto redisHGET = WFTaskFactory::create_redis_task(
    "redis://localhost", 0,
    [chkidx, req](WFRedisTask *t) {
        // 获取 filehash
        protocol::RedisValue val;
        t->get_resp()->get_result(val);
        string filehash = val.string_value();

        // 创建 hash 目录, 写分块文件
        mkdir(filehash.c_str(), 0777);
        string path = filehash + "/" + chkidx;

        const void *body; size_t size;
        req->get_parsed_body(&body, &size);
        int fd = open(path.c_str(), O_RDWR | O_CREAT, 0666);
        write(fd, body, size);
        close(fd);
    });
redisHGET->get_req()->set_request("HGET", {uploadID, "filehash"});
series_of(serverTask)->push_back(redisHGET);

// 3. 记录上传进度到 Redis
auto redisHSET = WFTaskFactory::create_redis_task(
    "redis://localhost", 0, nullptr);
redisHSET->get_req()->set_request("HSET",
    {uploadID, "chkidx_" + chkidx, "1"});
series_of(serverTask)->push_back(redisHSET);

resp->append_output_body("OK");
}

// === 接口 3: 通知合并 ===
// POST /file/mupload/complete?uploadID=xxx
else if (path == "/file/mupload/complete" && method == "POST") {
    string uploadID = /* 解析 query */;

    // 从 Redis 获取全部上传进度
    auto redisTask = WFTaskFactory::create_redis_task(
        "redis://localhost", 0,
        [resp](WFRedisTask *t) {
            protocol::RedisValue val;
            t->get_resp()->get_result(val);

            int chunkcount = 0, chunknow = 0;
            for (int i = 0; i < val.arr_size(); i += 2) {
                string key = val.arr_at(i).string_value();
                string v = val.arr_at(i+1).string_value();
                if (key == "chunkcount")
                    chunkcount = stoi(v);
                else if (key.substr(0, 7) == "chkidx_")
                    ++chunknow;
            }
        });
}

```

```

        if (chunkcount == chunknow) {
            // ★ 此处触发合并操作（或将消息发到 RabbitMQ）
            resp->append_output_body("SUCCESS");
        } else {
            resp->append_output_body("FAIL");
        }
    });
    redisTask->get_req()->set_request("HGETALL", {uploadID});
    series_of(serverTask)->push_back(redisTask);
}

```

3.6 Redis 的 Docker 部署

```
docker run -d --name redis -p 6379:6379 redis:7.0
```

四、第二层：阿里云 OSS

4.1 为什么不用本地磁盘

本地磁盘	阿里云 OSS
机器宕机数据丢失	99.9999999% 可靠性 (9 个 9)
磁盘满了要手动扩容	无限扩容，按用量付费
需要自己做备份	自动三副本 + 跨区域复制
带宽有限，多人下载卡死	CDN 加速，海量带宽

4.2 OSS 核心概念

AccessKey (AK)	- 身份凭证，不要泄露
Bucket	- 存储空间（类似一个超级大文件夹）
Object	- 文件（key = 文件名，value = 文件内容）
Endpoint	- 访问域名（内网/外网不同）

Region - 数据中心物理位置 (选离用户最近的)

4.3 OSS C++ SDK 核心代码

```
#include <alibabacloud/oss/OssClient.h>
using namespace AlibabaCloud::OSS;

// === 配置信息 (实际项目中放配置文件) ===
struct OSSInfo {
    string Bucket          = "my-cloud-disk-bucket";
    string EndPoint        = "oss-cn-hangzhou.aliyuncs.com";
    string AccessKeyID     = "your-access-key-id";
    string AccessKeySecret = "your-access-key-secret";
};

// === 初始化 (进程启动时执行一次) ===
InitializeSdk();

OSSInfo info;
ClientConfiguration conf; // 可配超时、重试、连接池
OssClient client(info.EndPoint, info.AccessKeyID,
                 info.AccessKeySecret, conf);

// === 上传文件到 OSS ===
string localPath = "tmp/user_video.mp4";
string ossKey    = "files/user123/video.mp4"; // OSS 上的路径

auto outcome = client.PutObject(info.Bucket, ossKey, localPath);

if (outcome.IsSuccess()) {
    cout << "Upload OK, ETag: " << outcome.result().ETag() << endl;
} else {
    cerr << "Upload FAIL: " << outcome.error().Message() << endl;
}

// === 生成下载链接 (有时效性) ===
time_t expireTime = time(nullptr) + 3600; // 1小时后过期
auto urlOutcome = client.GeneratePresignedUrl(
    info.Bucket, ossKey, expireTime, Http::Get);

if (urlOutcome.IsSuccess()) {
    string downloadUrl = urlOutcome.result(); // 直接给浏览器
}

// === 退出时释放资源 ===
ShutdownSdk();
```

4.4 编译链接 OSS SDK

```
g++ -std=c++11 -fno-rtti \  
    -lalibabacloud-oss-cpp-sdk \  
    -lcurl -lcrypto -lpthread \  
    main.cc -o server
```

4.5 OSS 集成到项目中的设计

文件上传流程 (加入 OSS 之后) :

用户上传 → C++ 服务接收 → 写本地 tmp/

- 计算 SHA1 → INSERT 元数据到 MySQL
- 异步推 OSS (PutObject)
- 删除本地 tmp 文件 (节省磁盘)

文件下载流程 (加入 OSS 之后) :

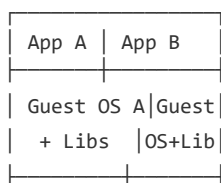
用户请求 → C++ 服务查 MySQL 元数据

- 调用 GeneratePresignedUrl 生成临时链接
- 返回链接给前端 → 前端直接从 OSS 下载

五、第三层 : Docker 容器化

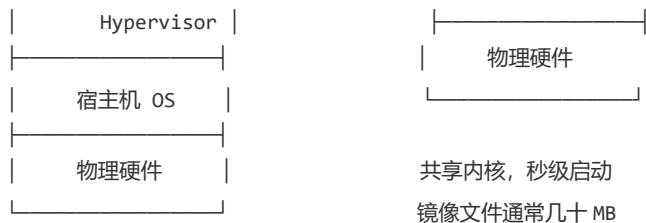
5.1 容器 vs 虚拟机

虚拟机架构:



Docker 架构:





虚拟化整个 OS，启动慢
镜像文件 GB 级别

5.2 Docker 关键命令速查

— 镜像相关 —

```
docker images          # 列出本地镜像
docker pull nginx:1.21 # 拉取镜像
docker rmi <image_id>  # 删除镜像
```

— 容器相关 —

```
docker run -d -p 8080:80 --name mynginx nginx      # 后台运行，端口映射
docker run -it ubuntu /bin/bash                    # 交互式进入容器
docker run -d -v /host/path:/container/path mysql:8.0 # 挂载数据卷
docker ps                                           # 运行中的容器
docker ps -a                                       # 所有容器
docker stop <container>                           # 停止
docker start <container>                           # 启动
docker rm <container>                              # 删除
docker exec -it <container> /bin/bash              # 进入运行中的容器
docker logs <container>                           # 查看日志
```

— 制作镜像 —

```
docker commit <container> myname:v1 # 从容器制作镜像
```

5.3 网盘项目的 Docker 部署方案

1. 启动 MySQL

```
docker run -d --name mysql -p 3306:3306 \
-e MYSQL_ROOT_PASSWORD=123 \
-v /data/mysql:/var/lib/mysql \
```

```
mysql:8.0

# 2. 启动 Redis
docker run -d --name redis -p 6379:6379 \
redis:7.0

# 3. 启动 RabbitMQ
docker run -d --name rabbit --hostname rabbitsvr \
-p 5672:5672 -p 15672:15672 \
-v /data/rabbitmq:/var/lib/rabbitmq \
rabbitmq:management

# 4. 启动 Consul (注册中心)
docker run -d --name consul -p 8500:8500 \
hashicorp/consul agent -server -bootstrap -ui \
-bind=0.0.0.0 -client=0.0.0.0

# 5. 启动自己的 C++ 服务
docker run -d --name fmm-service -p 2000:2000 \
-v /path/to/project:/app \
myclouddisk/fmm:latest
```

5.4 写一个 Dockerfile (示例)

```
FROM ubuntu:22.04

RUN apt update && apt install -y \
    build-essential cmake libssl-dev \
    libcurl4-openssl-dev libsnappy-dev liblz4-dev

COPY . /app
WORKDIR /app/build
RUN cmake .. && make -j$(nproc)

EXPOSE 2000
CMD ["/FMMService"]
```

六、第四层：RabbitMQ 消息队列

6.1 为什么需要消息队列

场景：文件上传成功后，需要同时做三件事：

1. 写元数据到 MySQL ─┐
2. 同步到 OSS ─┬─ 如果同步调用，上传接口要等这三件事全部完成才返回
3. 通知用户 ─┘

用消息队列后：

上传接口 → 发一条消息到队列 → 立即返回 "上传成功"

三个消费者各自从队列取消息 → 异步处理

好处：

- 响应变快（不用等 OSS 同步完成）
- 削峰（高峰时消息排队，不会打爆数据库）
- 解耦（上传服务不需要知道 OSS 的存在）

6.2 AMQP 核心概念

生产者 → Exchange(交换机) → Queue(队列) → 消费者

- ├─ Direct Exchange: 路由键完全匹配 → 单播
- ├─ Fanout Exchange: 广播给所有绑定的队列
- └─ Topic Exchange: 路由键模式匹配（如 "oss.*"）

6.3 C++ 使用 RabbitMQ (SimpleAmqpClient)

```
#include <SimpleAmqpClient/SimpleAmqpClient.h>
```

```
struct RabbitMQInfo {  
    string rabbitUrl = "amqp://guest:guest@127.0.0.1:5672";  
    string exchange = "uploadserver.trans";  
    string queue = "uploadserver.trans.oss";  
    string routingKey = "oss";  
};
```

```
// === 消息生产者（文件上传服务） ===
```

```
void publishUploadEvent(const string &filehash, const string &ossKey) {  
    auto channel = AmqpClient::Channel::Create(mqinfo.rabbitUrl);
```

```
    // 构造消息（JSON 格式）
```

```

    Json msg;
    msg["filehash"] = filehash;
    msg["ossKey"] = ossKey;

    auto message = AmqpClient::BasicMessage::Create(msg.dump());
    channel->BasicPublish(mqinfo.exchange, mqinfo.routingKey, message);
}

// === 消息消费者 (OSS 同步服务) ===
void consumeUploadEvents() {
    auto channel = AmqpClient::Channel::Create(mqinfo.rabbitUrl);
    channel->BasicConsume(mqinfo.queue, mqinfo.queue);

    while (true) {
        AmqpClient::Envelope::ptr_t envelope;
        bool hasMsg = channel->BasicConsumeMessage(envelope, 10000); // 10 秒超时

        if (hasMsg) {
            string body = envelope->Message()->Body();
            // 解析 JSON, 执行 OSS 备份...
        }
    }
}

```

6.4 网盘项目中使用消息队列的场景

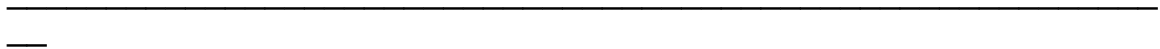
场景 1: 文件上传 → 异步备份到 OSS

上传服务 → 发消息到 exchange "uploadserver.trans"
 → routingKey "oss"
 → OSS 备份服务消费

场景 2: 文件删除 → 异步清理 OSS + 通知相关用户

删除服务 → 发消息
 → OSS 清理服务消费
 → 通知服务消费

场景 3: 文件分享 → 异步通知被分享者



七、第五层：protobuf + SRPC

7.1 从 REST 到 RPC

REST 风格 (v1 使用)：

客户端：POST http://ip/user/signin Body: {"user":"jack","pwd":"123"}

服务端：解析 HTTP → 解析 JSON → 调函数 → 返回 JSON

问题：手工构造/解析 HTTP, JSON 序列化效率低

RPC 风格 (完全体使用)：

客户端：SignupResult result = UserService::Signup("jack", "123");

服务端：跟本地函数调用写法一模一样

框架自动处理：网络传输 + 序列化 + 服务发现 + 负载均衡

7.2 protobuf IDL 设计

```
// 文件名: cloudisk.proto
syntax = "proto3";

// ===== 用户服务 =====
message ReqSignup {
    string username = 1;
    string password = 2;
}

message ResSignup {
    int32 code = 1; // 0 成功, 非0 失败
    string message = 2;
}

// ===== 文件上传服务 =====
message ReqUploadInit {
    string filename = 1;
    int64 filesize = 2;
    string filehash = 3;
}

message ResUploadInit {
    string uploadID = 1;
    int32 chunkcount = 2;
    int32 chunksize = 3;
}

message ReqUploadPart {
```

```

        string uploadID = 1;
        int32  chkidx   = 2;
        bytes  data     = 3;
    }

    message RespUploadPart {
        int32 code = 1;
    }

    message ReqUploadComplete {
        string uploadID = 1;
    }

    message RespUploadComplete {
        int32 code = 1;
        string fileUrl = 2; // OSS 下载地址
    }

    // ===== 文件列表查询 =====
    message ReqFileList {
        string username = 1;
        string token    = 2;
        int32  limit     = 3;
    }

    message FileInfo {
        string filehash = 1;
        string filename = 2;
        int64  filesize = 3;
        string uploadAt = 4;
        string lastUpdated = 5;
    }

    message RespFileList {
        int32 code = 1;
        repeated FileInfo files = 2; // repeated = 数组
    }

    // ===== 服务定义 =====
    service CloudiskUser {
        rpc Signup(ReqSignup) returns (RespSignup);
        rpc Signin(ReqSignin) returns (RespSignin);
        rpc GetUserInfo(ReqUserInfo) returns (RespUserInfo);
    }

    service CloudiskFile {
        rpc UploadInit(ReqUploadInit) returns (RespUploadInit);
        rpc UploadPart(ReqUploadPart) returns (RespUploadPart);
        rpc UploadComplete(ReqUploadComplete) returns (RespUploadComplete);
        rpc FileList(ReqFileList) returns (RespFileList);
    }

```

7.3 生成 C++ 代码

```
protoc cloudisk.proto --cpp_out=. --proto_path=.  
# 生成: cloudisk.pb.h   cloudisk.pb.cc
```

7.4 使用生成的代码

```
#include "cloudisk.pb.h"  
  
// === 序列化 (客户端发送前) ===  
ReqSignup req;  
req.set_username("jack");  
req.set_password("123");  
  
string serialized;  
req.SerializeToString(&serialized); // 二进制, 比 JSON 小很多  
  
// 通过网络发送 serialized...  
  
// === 反序列化 (服务端收到后) ===  
ReqSignup req2;  
req2.ParseFromString(serialized);  
cout << req2.username() << endl; // "jack"  
cout << req2.password() << endl; // "123"  
  
// === 构造响应 ===  
RespSignup resp;  
resp.set_code(0);  
resp.set_message("注册成功");  
string respStr;  
resp.SerializeToString(&respStr);  
// 发送 respStr 给客户端
```

7.5 protobuf 的兼容性保证

向前兼容 (老代码读新消息) :

新增字段 → 用新编号 → 老代码遇到未知编号直接跳过

向后兼容 (新代码读老消息) :

新增字段必须标 optional → 没有也不报错

规则：

1. 不能修改已有字段的编号（改了就乱了）
2. 可以改字段名（编号不变就行）
3. 可以安全地增加 optional 字段

7.6 SRPC — 基于 Workflow 的 RPC 框架

SRPC 是搜狗开源的 RPC 框架，天然集成 Workflow，支持 protobuf IDL。

```
# 安装依赖
sudo apt install liblz4-dev libsnappy-dev

# 编译安装 SRPC
cd src-0.10.2
mkdir build && cd build
cmake .. && make && sudo make install
```

SRPC 使用示例（服务端 + 客户端由同一个 IDL 文件自动生成存根）：

```
// 1. 先写 IDL (cloudisk.proto, 如上)
// 2. protoc 生成 .pb.h / .pb.cc
// 3. SRPC 的 protoc 插件生成 server_stub 和 client_stub

// === 服务端代码 ===
class UserServiceImpl : public CloudiskUser::Service {
    void Signup(ReqSignup *req, RespSignup *resp, srpc::RPCContext *ctx) override {
        string username = req->username();
        string password = req->password();
        // ... 业务逻辑: 加密密码、写 MySQL ...
        resp->set_code(0);
        resp->set_message("注册成功");
    }
};

int main() {
    SRPCServer server;
    UserServiceImpl impl;
    server.add_service(&impl);
    server.start(2000); // 监听 2000 端口
```

```
        waitGroup.wait();
    }

// === 客户端代码 ===
int main() {
    CloudiskUser::SRPCClient client("127.0.0.1", 2000);

    ReqSignup req;
    req.set_username("jack");
    req.set_password("123");

    RespSignup resp;
    client.Signup(&req, &resp);

    cout << "code: " << resp.code() << ", msg: " << resp.message() << endl;
}
```

八、第六层：Consul 服务注册发现

8.1 为什么需要注册中心

问题：微服务有多个实例，客户端的 IP 写死在代码里？

方案 A：写死 IP — 实例挂了调用失败

方案 B：Nginx 反向代理 — Nginx 本身也是单点

方案 C：Consul 注册中心 — 每个实例自动注册，客户端动态发现

Consul 的工作原理：

1. FMM 服务启动 → 向 Consul 注册 "FMM" / IP:port / 健康检查 URL
2. 客户端调用 → 先问 Consul "FMM 有哪些可用实例？"
3. Consul 返回健康的实例列表 → 客户端选一个调用
4. 某实例挂了 → Consul 健康检查失败 → 从列表移除

8.2 Consul 集群部署

```
# 节点 1 (bootstrap, 2 节点集群)
docker run -d --name consul1 -p 8500:8500 \
  hashicorp/consul agent -server -bootstrap-expect=2 \
  -ui -bind=0.0.0.0 -client=0.0.0.0

# 获取节点 1 的 IP
docker inspect --format='{{.NetworkSettings.IPAddress}}' consul1
# 假设输出: 172.17.0.3

# 节点 2 和 3 加入节点 1
docker run -d --name consul2 -p 8501:8500 \
  hashicorp/consul agent -server -ui \
  -bind=0.0.0.0 -client=0.0.0.0 -join 172.17.0.3

docker run -d --name consul3 -p 8502:8500 \
  hashicorp/consul agent -server -ui \
  -bind=0.0.0.0 -client=0.0.0.0 -join 172.17.0.3
```

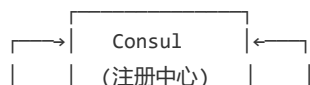
8.3 C++ 接入 Consul (ppconsul 库)

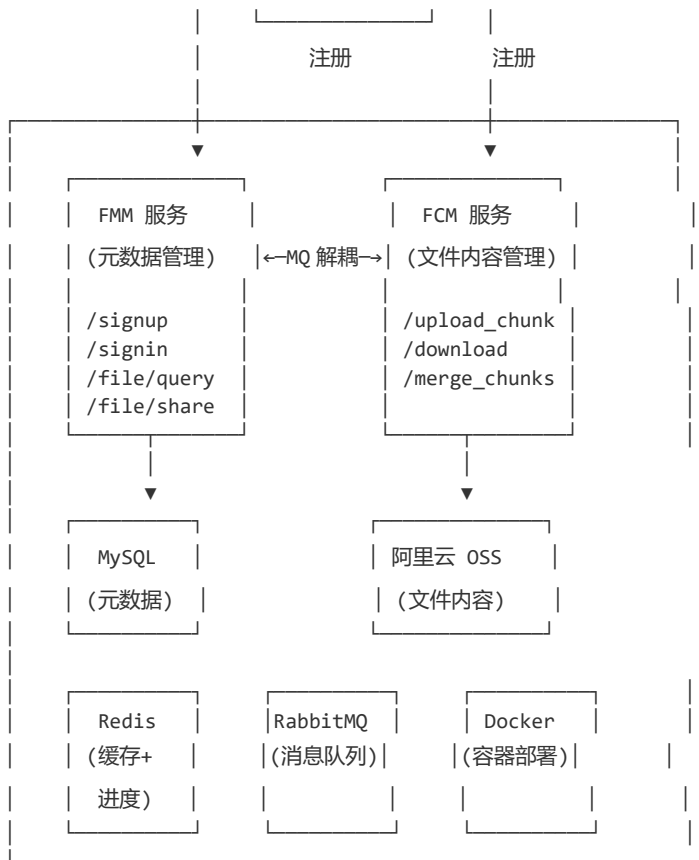
```
// ppconsul 是一个 C++ Consul 客户端库
// 服务注册、健康检查、KV 存储都可以通过它调用

// 伪代码: 服务注册
ConsulClient consul("http://127.0.0.1:8500");
consul.registerService("FMM-Service", "192.168.1.100", 2000,
  "/health"); // Consul 会定期 GET /health 检查

// 伪代码: 服务发现
auto instances = consul.discoverService("FMM-Service");
// 返回 [{ip: "192.168.1.100", port: 2000, healthy: true}, ...]
// 客户端从中随机/轮询选一个调用
```

九、完全体架构全景





请求流程示例：用户上传 500MB 视频

1. 客户端 → Consul → 获取 FMM 实例列表 → 选一个
2. 客户端 → FMM.UploadInit("video.mp4", 500MB, SHA1)
3. FMM → MySQL 查询是否已存在 (去重)
4. FMM → Redis 初始化上传进度 (HSET uploadID chunkcount=500...)
5. FMM → 返回 {uploadID, chunkcount=500, chunksize=1MB}
6. 客户端 并发请求 → FCM.UploadPart(uploadID, chunkid, data) × N
7. FCM → Redis 查 filehash → 写分块到本地 → HSET 记录进度
8. 客户端 全部传完 → FMM.UploadComplete(uploadID)
9. FMM → Redis HGETALL 验证进度 → 触发合并
10. FMM → 发消息到 RabbitMQ → {event: "file_ready", filehash: "xxx"}
11. OSS 备份服务 (消费者) → 合并分块 → PutObject 到 OSS
12. FMM → MySQL 插入元数据记录

各组件的作用总结

组件	解决什么问题	v1 有吗
Redis	分块上传进度追踪、Token 缓存	无
阿里云 OSS	文件可靠存储、无限扩容	无（本地 tmp/）
Docker	环境一致性、一键部署	无
RabbitMQ	异步解耦、削峰	无（同步写）
protobuf	高效序列化、接口自动生成	无（JSON）
SRPC	远程过程调用（像调本地函数）	无（HTTP REST）
Consul	服务发现 + 健康检查 + 负载均衡	无（单点）

十、毕设级工程实施路线图

阶段 1：搭建基础设施（1-2 天）

- 任务 1.1 用 Docker 启动 MySQL、Redis、RabbitMQ、Consul
- 任务 1.2 编译安装 protobuf、SRPC、SimpleAmqpClient、aliyun-oss-cpp-sdk
- 任务 1.3 创建一个 C++ 项目模板，写 CMakeLists.txt 链接所有库

验证标准： 在 C++ 代码中能分别连接上每个组件。

阶段 2：单体分块上传（2-3 天）

- 任务 2.1 在 v1 的 CloudiskServer 上加分块上传三个接口
- 任务 2.2 用 Redis 存储上传进度
- 任务 2.3 写前端页面（或 curl 脚本）测试完整流程

验证标准： 上传一个 100MB 文件，模拟断网（关掉客户端），重新上传，确认只有未完成的分块被重传。

阶段 3：OSS 集成（1 天）

任务 3.1 注册阿里云 OSS，获取 AccessKey

任务 3.2 在分块合并完成后，异步上传合并后的文件到 OSS

任务 3.3 下载接口改为返回 OSS 临时下载链接

验证标准： 上传的文件出现在 OSS 控制台，下载链接能正常访问。

阶段 4：消息队列解耦（1-2 天）

任务 4.1 用 RabbitMQ 替代直接的 OSS 上传调用

任务 4.2 写独立的 OSS 备份消费者进程

任务 4.3 加入重试和死信队列

验证标准： 上传完成后，即使 OSS 消费者没启动，消息不会丢失；消费者启动后自动处理积压消息。

阶段 5：拆分微服务（2-3 天）

任务 5.1 设计 protobuf IDL 文件（用户服务 + 文件服务）

任务 5.2 把 v1 的 CloudiskServer 拆成两个独立进程：

- FMM 服务（用户注册登录、文件元数据查询、分享）
- FCM 服务（文件分块上传、合并、OSS 同步）

任务 5.3 用 SRPC 实现 FMM 和 FCM 之间的 RPC 调用

验证标准： 三个独立进程各自独立启动，通过 SRPC 调用能跑通完整的上传 → 下载 → 分享流程。

阶段 6：服务注册发现（1 天）

任务 6.1 FMM 和 FCM 启动时向 Consul 注册

任务 6.2 客户端调用前先从 Consul 发现可用实例

任务 6.3 加入健康检查

验证标准： 手动 kill 一个 FCM 实例，Consul 自动移除，客户端自动切换到另一个实例。

阶段 7：限速与 VIP（1 天）

任务 7.1 在 User 表加 user_type 字段 (0=普通, 1=VIP)

任务 7.2 FMM 根据 user_type 返回不同数量的 FCM 节点列表
(VIP: 10 个节点, 普通: 2 个节点)

阶段 8：文件分享与 RBAC（2-3 天）

任务 8.1 设计 Role 表、UserRole 表、Permission 表

任务 8.2 实现好友分享 (插入 Permission, 指定 user_id)

任务 8.3 实现链接分享 (插入 Permission, user_id=NULL)

任务 8.4 定时清理过期权限 (过期时间字段)

阶段 9：Docker Compose 编排（1 天）

```
# docker-compose.yml
version: '3'
services:
```

```
consul:
  image: hashicorp/consul:latest
  ports: ["8500:8500"]

mysql:
  image: mysql:8.0
  environment:
    MYSQL_ROOT_PASSWORD: "123"
  volumes: ["/data/mysql:/var/lib/mysql"]

redis:
  image: redis:7.0

rabbitmq:
  image: rabbitmq:management
  ports: ["5672:5672", "15672:15672"]

fmm:
  build: ./fmm-service
  ports: ["2000:2000"]
  depends_on: [mysql, consul]

fcm:
  build: ./fcm-service
  ports: ["2001:2001"]
  depends_on: [redis, rabbitmq, consul]

# 一键启动所有服务
# docker-compose up -d
```

十一、关键技术决策与对比

11.1 为什么用 protobuf 而不是 JSON

	JSON	protobuf
序列化后大小	~100%	~30% (二进制编码)
解析速度	慢 (文本解析)	快 (二进制读取)
类型安全	弱 (都是字符串)	强 (int32/int64/bytes 严格区分)
向前后兼容	需要手动处理	原生支持
可读性	人眼可读	不可读 (需要工具)
自动生成存根	无	有 (protoc 生成)

11.2 为什么用 SRPC 而不是 gRPC

	gRPC	SRPC
底层引擎	自研	搜狗 Workflow (和 v1 同源)
学习曲线	中等	低 (Workflow 用户零成本)
异步性能	回调式	Series 任务链 (更直观)
中文文档	少	完善

11.3 为什么用 RabbitMQ 而不是 Kafka

	RabbitMQ	Kafka
协议	AMQP (标准协议)	私有协议
适合场景	业务消息、RPC 回调	海量日志、流处理
消息优先级	支持	不支持
部署难度	低 (单节点即可)	高 (需要 ZK)
本场景	文件上传事件通知 (小消息量)	不适用

十二、可用资源清单

在 C:\Users\fengx\Desktop\王道 c++\6_workflow\课程资料\ 下可直接使用：

文件	用途
`workflow-0.11.3.tar.gz`	异步引擎 (v1 已用)
`wfrest-0.9.5.tar.gz`	HTTP REST 框架 (v1 已用)
`protobuf-3.20.1.tar.gz`	序列化框架
`srpc-0.10.2.tar.gz`	RPC 框架 (基于 Workflow)
`aliyun-oss-cpp-sdk-1.10.0.tar.gz`	阿里云 OSS C++ SDK

`rabbitmq-c-0.11.0.tar.gz`	RabbitMQ C 客户端
`SimpleAmqpClient-2.5.1.tar.gz`	RabbitMQ C++ 封装（更易用）
`ppconsul-0.2.3.tar.gz`	Consul C++ 客户端
`lz4-1.9.4.tar.gz`	压缩库（SRPC 依赖）
`tinysql.10.0.0.tar.gz`	XML 解析库（某些依赖需要）
`分块上传答案.md`	分块上传完整参考代码
`04_微服务相关的技术.pdf`	微服务各组件详细教程
`05_网盘项目完全体.pdf`	完全体架构设计文档

附录：学习检查清单

完成以下每一步，打勾确认理解：

- [] 能画出 v1 单机版的架构图和数据流
- [] 能手写一个 Workflow Series 任务链（MySQL 查询 → 处理结果 → 写回）
- [] 能解释 Lambda 值捕获 [=] 和引用捕获 [&] 在异步场景下的区别
- [] 能在 Docker 中启动 Redis，并用 C++ 读写
- [] 能解释分块上传的三个接口和 Redis 进度追踪机制
- [] 能用阿里云 OSS SDK 上传文件并生成下载链接
- [] 能写一个 protobuf IDL 文件，包含 message 和 service 定义
- [] 能解释 protobuf 向前兼容和向后兼容的实现原理
- [] 能用 RabbitMQ 实现一个生产者 → 消费者的异步流程
- [] 能画出完全体架构图，标注每个组件的职责

- [] 能解释为什么文件元数据和文件内容要分离存储
- [] 能讲清楚 RBAC 权限控制的表设计和分享流程
- [] 能写 docker-compose.yml 一键启动整个系统